
Aprendizaje profundo utilizando redes neuronales multicapa

El aprendizaje profundo es la tendencia más reciente en aprendizaje automático/IA. Se centra en la construcción de redes neuronales avanzadas. Al integrar múltiples capas ocultas en un modelo de red neuronal, podemos trabajar con representaciones no lineales complejas de datos.

Creemos aprendizaje profundo utilizando redes neuronales base. El aprendizaje profundo tiene numerosos casos de uso en la vida real, como vehículos autónomos, diagnósticos médicos, visión artificial, reconocimiento de voz, procesamiento del lenguaje natural (PLN), reconocimiento de escritura a mano, traducción de idiomas y muchos otros campos.

En este documento, abordaremos el proceso de aprendizaje profundo: cómo entrenar, probar e implementar una red neuronal profunda (DNN, *Deep Neural Network*). Analizaremos los diferentes paquetes disponibles en R para gestionar DNN. Comprenderemos cómo construir y entrenar una DNN con el paquete `neuralnet`. Finalmente, analizaremos un ejemplo de entrenamiento y modelado de una DNN utilizando `h2o`, la plataforma escalable de aprendizaje de memoria abierta, para crear modelos con grandes conjuntos de datos e implementar predicciones con métodos de alta precisión.

Los siguientes son los temas que se abordan en este capítulo:

- Tipos de DNN
- Paquetes de R para aprendizaje profundo
- Entrenamiento y modelado de una DNN con `neuralnet`
- La biblioteca `h2o`

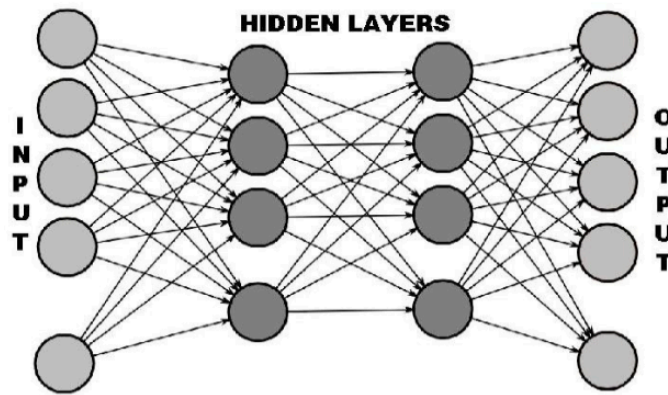
Al final de este documento, comprenderemos los conceptos básicos del aprendizaje profundo y cómo implementarlo en el entorno R. Descubriremos diferentes tipos de DNN. Aprenderemos a entrenar, probar e implementar un modelo. Sabremos cómo entrenar y modelar una DNN con `h2o`.

Introducción a las DNN

Con la llegada de la infraestructura de procesamiento de big data, la GPU y la GP-GPU, ahora podemos superar los desafíos de las redes neuronales superficiales, como el sobreajuste y el gradiente de desaparición, utilizando diversas funciones de activación y técnicas de

regularización L1/L2. El aprendizaje profundo puede procesar grandes cantidades de datos, etiquetados y no etiquetados, de forma fácil y eficiente.

Como se mencionó, el aprendizaje profundo es una clase de aprendizaje automático en la que el aprendizaje se produce en múltiples niveles de redes neuronales. El diagrama estándar que representa una red neuronal profunda (DNN) se muestra en la siguiente figura:



Del análisis de la figura anterior, podemos observar una analogía notable con las redes neuronales que hemos estudiado hasta ahora. Podemos estar tranquilos: a diferencia de lo que podría parecer, el aprendizaje profundo es simplemente una extensión de la red neuronal.

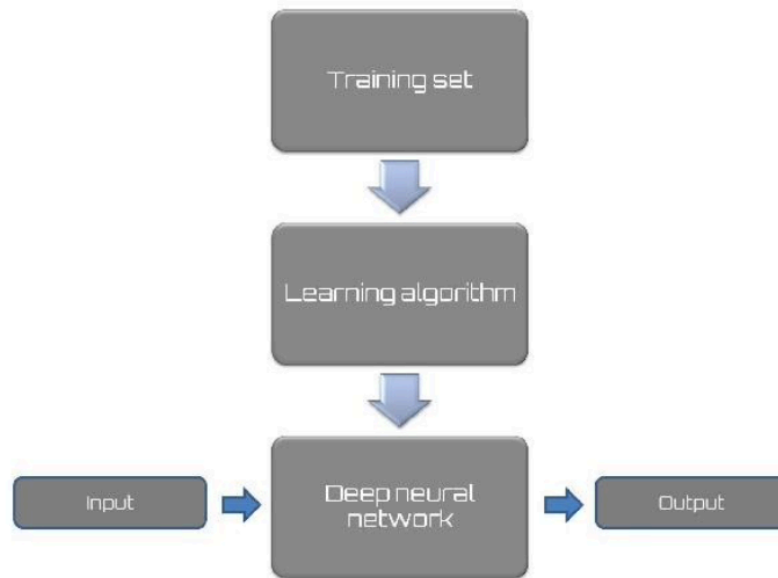
En este sentido, gran parte de lo visto en los temas anteriores es válido. En resumen, una DNN es una red neuronal multicapa que contiene dos o más capas ocultas. Nada complicado. Al añadir más capas y más neuronas por capa, aumentamos la especialización del modelo para entrenar datos, pero disminuimos el rendimiento con los datos de prueba.

Como anticipamos, las DNN son derivadas de las ANN (*Artificial Neural Networks*). Al aumentar el número de capas ocultas a más de una, construimos las DNN. Existen muchas variantes de DNN, como lo ilustran los diferentes términos que se muestran a continuación:

- **Red de creencias profundas** (DBN, *Deep Belief Network*): Normalmente es una red de propagación hacia adelante en la que los datos fluyen de una capa a otra sin retorno. Hay al menos una capa oculta, pero puede haber varias, lo que aumenta la complejidad.
- **Máquina de Boltzmann restringida** (RBM, *Restricted Boltzmann Machine*): Tiene una sola capa oculta y no existe conexión entre los nodos de un grupo. Es un modelo MLP (**Multi Layer Perceptron**) simple de redes neuronales.

- **Redes neuronales recurrentes** (RNN, *Recurrent Neural Networks*) y **Memoria a largo plazo** (LSTM, *Long Short Term Memory*): Estas redes permiten que los datos fluyan en cualquier dirección dentro y entre grupos.

Como cualquier algoritmo de aprendizaje automático, incluso las DNN requieren procesos de desarrollo, entrenamiento y evaluación. La siguiente figura muestra un flujo de trabajo básico para el aprendizaje profundo:



El flujo de trabajo que vimos en la figura anterior es muy similar al típico de un algoritmo de aprendizaje supervisado. Pero ¿qué lo diferencia de otros algoritmos de aprendizaje automático?

Casi todos los algoritmos de aprendizaje automático presentan limitaciones a la hora de identificar las características de los datos de entrada sin procesar, especialmente cuando son complejos y carecen de un orden aparente, como en el caso de las imágenes. Normalmente, esta limitación se supera con la ayuda de las personas, que se encargan de identificar lo que la máquina no puede hacer. El aprendizaje profundo elimina este paso, basándose en el proceso de entrenamiento para encontrar los modelos más útiles a partir de ejemplos de entrada.

Incluso en este caso, la intervención humana es necesaria para tomar decisiones antes de comenzar el entrenamiento, pero el descubrimiento automático de características simplifica enormemente la tarea. Lo que hace que las redes neuronales sean particularmente ventajosas, en comparación con otras soluciones que ofrece el aprendizaje automático, es su gran capacidad de generalización.

Estas características han hecho que el aprendizaje profundo sea muy eficaz para casi todas las tareas que requieren aprendizaje automático; aunque **es particularmente eficaz en el caso de datos jerárquicos complejos**. Su base subyacente de ANN (Artificial Neural Network) forma representaciones altamente no lineales; estas suelen estar compuestas por múltiples capas junto con transformaciones no lineales y arquitecturas personalizadas.

En esencia, el aprendizaje profundo funciona muy bien con datos desordenados del mundo real, lo que lo convierte en una herramienta clave en varios campos tecnológicos de los próximos años. Hasta hace poco, era un área difícil de comprender, pero su éxito ha generado numerosos recursos y proyectos excelentes que facilitan más que nunca comenzar.

Ahora que sabemos qué son las DNN, veamos qué herramientas nos ofrece el entorno de desarrollo R para abordar este tema en particular.

R para DNN

En la sección anterior, aclaramos algunos conceptos clave que fundamentan el aprendizaje profundo. También comprendimos las características que hacen que su uso sea particularmente conveniente. Además, su rápida difusión se debe a la gran disponibilidad de una amplia gama de frameworks y bibliotecas para diversos lenguajes de programación.

El lenguaje de programación R es ampliamente utilizado por científicos y programadores gracias a su extrema facilidad de uso. Además, existe una extensa colección de bibliotecas que permiten la visualización y el análisis profesional de datos con los algoritmos más populares.

La rápida difusión de los algoritmos de aprendizaje profundo ha dado lugar a la creación de un número cada vez mayor de paquetes disponibles para este fin, incluso en R.

La siguiente tabla muestra los distintos paquetes/interfaces disponibles para el **aprendizaje profundo** con R:

Paquete CRAN	Taxonomía de redes neuronales compatible	Lenguaje subyacente/proveedor
MXNet	Feed-forward, CNN	CC++/CUDA
darch	RBM, DBN	C/C++
deepnet	Feed-forward, RBM, DBN, autoencoders	R
h2o	Feed-forward network, autoencoders	Java
nnet y neuralnet	Feed-forward	R
Keras	Variedad de DNNs	Python/keras.io
TensorFlow	Variedad de DNNs	C++/Python/Google

MXNet es una biblioteca de aprendizaje profundo moderna y portátil compatible con múltiples máquinas. Las empresas y universidades más grandes del mundo han adoptado MXNet como framework de aprendizaje automático. Entre ellas se incluyen Amazon, Intel, Data, Baidu, Microsoft, Wolfram Research, Carnegie Mellon, MIT, la Universidad de Washington y la Universidad de Ciencia y Tecnología de Hong Kong.

MXNet es un framework de código abierto que permite un modelado rápido y admite un modelo de programación flexible en múltiples lenguajes (C++, Python, Julia, MATLAB, JavaScript, Go, R, Scala, Perl y Wolfram Language).

El framework MXNet es compatible con el lenguaje de programación R. El paquete MXNet R proporciona computación GPU flexible y eficiente, además de una profundización de vanguardia en R. Permite escribir cálculos tensoriales/matriciales fluidos con múltiples GPU en R. También permite construir y personalizar modelos de aprendizaje profundo de vanguardia en R y aplicarlos a actividades como la clasificación de imágenes y los desafíos de la ciencia de datos.

El framework darch se basa en el código escrito por G. E. Hinton y R. R. Salakhutdinov, y está disponible en el entorno MATLAB para DBN. Este paquete puede generar redes neuronales con múltiples niveles (arquitecturas profundas) y formarlas con un método innovador desarrollado por los autores. Este método proporciona una preformación con el **método de divergencia** contrastante publicado por G. Hinton (2002) y un ajuste fino con algoritmos de entrenamiento comunes conocidos como **retropropagación o gradientes conjugados**. Además, la supervisión del ajuste fino puede mejorarse con maxout y dropout, dos técnicas desarrolladas recientemente para mejorar el ajuste fino para el aprendizaje profundo.

La biblioteca deepnet es un paquete relativamente pequeño, pero bastante potente, con una variedad de arquitecturas para elegir. Esta biblioteca implementa algunas arquitecturas de aprendizaje profundo y algoritmos de redes neuronales, incluyendo retropropagación, RBM, DBN, autocodificador profundo, etc. A diferencia de las otras bibliotecas que hemos analizado, fue escrita específicamente para R. Tiene varias funciones, incluyendo:

- `nn.train`: Para entrenar redes neuronales de una o varias capas ocultas mediante BP (Back-Propagation)
- `nn.predict`: Para predecir nuevas muestras mediante la red neuronal entrenada
- `dbn.dnn.train`: Para entrenar una DNN con pesos inicializados por DBN
- `rbm.train`: Para entrenar una RBM

El paquete `h2o` de R incluye funciones para construir regresión lineal general, K-medias, Naive Bayes, Análisis de componentes principales (PCA), bosques y aprendizaje profundo (modelos de redes neuronales multicapa). `h2o` es un paquete externo a CRAN, está desarrollado en Java y está disponible para diversas plataformas. Es un motor matemático de código abierto para big data que calcula algoritmos de aprendizaje automático distribuido en paralelo.

Los paquetes `nnet` y `neuralnet` se han analizado ampliamente en los temas anteriores. Se trata de dos paquetes para la gestión de redes neuronales en R. También permiten construir y entrenar redes neuronales multinúcleo, por lo que se basan en el aprendizaje profundo.

Keras es una biblioteca de redes neuronales de código abierto escrita en Python. Diseñada para permitir una experimentación rápida con redes neuronales profundas (DNN), se centra en ser minimalista, modular y extensible. La biblioteca contiene numerosas implementaciones de componentes básicos de redes neuronales de uso común, como capas, objetivos, funciones de activación, optimizadores y una serie de herramientas para facilitar el trabajo con datos de imagen y texto. El código está alojado en GitHub, y los foros de soporte de la comunidad incluyen la página de problemas de GitHub, un canal de Gitter y un canal de Slack.

TensorFlow es una biblioteca de software de código abierto para aprendizaje automático. Contiene un sistema para **construir y entrenar redes neuronales para detectar y descifrar patrones y correlaciones**, con métodos similares a los adoptados por el aprendizaje humano. Se utiliza tanto para búsqueda como para producción de Google.

Redes neuronales multicapa con neuralnet

Tras comprender los fundamentos del aprendizaje profundo, es hora de aplicar las habilidades adquiridas a un caso práctico. En la sección anterior, vimos que dos bibliotecas que conocemos figuran en los paquetes disponibles en la sección de R para DNN. Me refiero a los paquetes `nnet` y `neuralnet` que aprendimos a usar en los temas anteriores mediante ejemplos prácticos. Dado que tenemos algo de práctica con la biblioteca `neuralnet`, creo que deberíamos comenzar nuestra exploración práctica del fascinante mundo del aprendizaje profundo desde aquí.

Para empezar, presentamos el conjunto de datos que utilizaremos para construir y entrenar la red. Se llama *College* dataset y contiene estadísticas de un gran número de universidades estadounidenses, recopiladas de la edición de 1995 de US News and World Report. Este conjunto de datos se obtuvo de la biblioteca StatLib, mantenida por la Universidad Carnegie Mellon.

Para nosotros, el proceso se simplifica aún más porque no tenemos que recuperar los datos ni importarlos a R, ya que están contenidos en un paquete de R. Me refiero al paquete ISLR. Solo tenemos que instalarlo y cargar la biblioteca correspondiente. Pero esto lo veremos más adelante, cuando expliquemos los códigos en detalle.

Ahora veamos el contenido del conjunto de datos College. Es un frame de datos con 777 observaciones sobre las siguientes 18 variables:

- Private: Un factor con niveles No y Yes que indican universidad privada o pública
- Apps: Número de solicitudes recibidas
- Accept: Número de solicitudes aceptadas
- Enroll: Número de nuevos estudiantes matriculados
- Top10perc: Porcentaje de nuevos estudiantes del 10% superior de la clase de preparatoria
- Top25perc: Porcentaje de nuevos estudiantes del 25% superior de la clase de preparatoria
- F.Undergrad: Número de estudiantes de pregrado a tiempo completo
- P. Undergrad: Número de estudiantes de pregrado a tiempo parcial
- Outstate: Matrícula para estudiantes fuera del estado
- Room.Board: Costos de alojamiento y manutención
- Books: Costos estimados de libros
- Personal: Gastos personales estimados
- PhD: Porcentaje de profesores con doctorado
- Terminal: Porcentaje de profesores con título terminal
- S.F.Ratio: Ratio estudiante-profesor
- perc.alumni: Porcentaje de exalumnos que donan
- Expend: Gasto educativo por estudiante
- Grad.Rate: Tasa de graduación

Nuestro objetivo será construir una red neuronal multicapa capaz de predecir si la institución educativa es pública o privada, con base en los valores asumidos por las otras 17 variables:

```
> library("neuralnet")
> library(ISLR)
> data = College
> str(data)

> max_data <- apply(data[,2:18], 2, max)
> min_data <- apply(data[,2:18], 2, min)
> data_scaled <- scale(data[,2:18], center = min_data, scale = max_data - min_data)
```

```

> Private = as.numeric(College$Private)-1
> data_scaled = cbind(Private,data_scaled)
> index = sample(1:nrow(data),round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])
> n = names(train_data)
> f <- as.formula(paste("Private ~", paste(n[!n %in% "Private"], collapse = " + ")))
> deep_net = neuralnet(f,data=train_data,hidden=c(5,3),linear.output=F)
> plot(deep_net)
> predicted_data <- compute(deep_net,test_data[,2:18])
> print(head(predicted_data$net.result))
> predicted_data$net.result <- sapply(predicted_data$net.result,round,digits=0)
> table(test_data$Private,predicted_data$net.result)

```

Como es habitual, analizaremos el código línea por línea, explicando en detalle todas las características aplicadas para capturar los resultados.

```

> library("neuralnet")
> library(ISLR)

```

Como es habitual, las dos primeras líneas del código inicial se utilizan para cargar las bibliotecas necesarias para ejecutar el análisis.

Recuerda que para instalar una biblioteca que no está presente en la distribución inicial de R, debes usar la función `install.package`.

Esta es la función principal para instalar paquetes. Toma un vector de nombres y una biblioteca de destino, descarga los paquetes de los repositorios y los instala. Esta función debe usarse solo una vez, no cada vez que se ejecuta el código.

```

> data = College
> str(data)

```

Este comando carga el conjunto de datos `College`, que, como anticipamos, está contenido en la biblioteca `ISLR`, y lo guarda en un dataframe determinado.

Usamos la función `str` que permite visualizar la estructura interna de un objeto. La siguiente figura muestra algunos de los datos del conjunto de datos `College`:


```
'data.frame': 777 obs. of 18 variables:
 $ Private : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 2 2 2 2 ...
 $ Apps : num 1660 2186 1428 417 193 ...
 $ Accept : num 1232 1924 1097 349 146 ...
 $ Enroll : num 721 512 336 137 55 158 103 489 227 172 ...
 $ Top10perc : num 23 16 22 60 16 38 17 37 30 21 ...
 $ Top25perc : num 52 29 50 89 44 62 45 68 63 44 ...
 $ F.Undergrad: num 2885 2683 1036 510 249 ...
 $ P.Undergrad: num 537 1227 99 63 869 ...
 $ Outstate : num 7440 12280 11250 12960 7560 ...
 $ Room.Board : num 3300 6450 3750 5450 4120 ...
 $ Books : num 450 750 400 450 800 500 500 450 300 660 ...
 $ Personal : num 2200 1500 1165 875 1500 ...
 $ PhD : num 70 29 53 92 76 67 90 89 79 40 ...
 $ Terminal : num 78 30 66 97 72 73 93 100 84 41 ...
 $ S.F.Ratio : num 18.1 12.2 12.9 7.7 11.9 9.4 11.5 13.7 11.3 11.5 ...
 $ perc.alumni: num 12 16 30 37 2 11 26 37 23 15 ...
 $ Expend : num 7041 10527 8735 19016 10922 ...
 $ Grad.Rate : num 60 56 54 59 15 55 63 73 80 52 ...
```

¿Cómo se puede observar que para cada universidad se lista una serie de estadísticas? Las filas representan las observaciones en las columnas, mientras que las características detectadas están presentes:

```
> max_data <- apply(data[,2:18], 2, max)
> min_data <- apply(data[,2:18], 2, min)
> data_scaled <- scale(data[,2:18], center = min_data, scale = max_data - min_data)
```

En este fragmento de código, necesitamos normalizar los datos.

Recuerda que es recomendable normalizar los datos antes de entrenar una red neuronal. Con la normalización, se eliminan las unidades de datos, lo que permite comparar fácilmente datos de diferentes ubicaciones.

Para este ejemplo, utilizaremos el método **min-max** (generalmente llamado **escalado de características**) para obtener todos los datos escalados en el rango [0,1]. Antes de aplicar el método elegido para la normalización, se deben calcular los valores mínimo y máximo de cada columna de la base de datos. Este procedimiento ya se adoptó previamente en ejemplos de los temas iniciales, proceso de aprendizaje en redes neuronales.

La última línea escala los datos adoptando la regla de normalización esperada. Ten en cuenta que **realizamos la normalización solo en las últimas 17 filas (de la 2 a la 18), excluyendo la primera columna, Private**, que contiene un factor con niveles "No" y "Yes", que indica si la universidad es privada o pública. Esta variable será nuestro objetivo en la red que vamos a construir. Para confirmar lo que decimos, revisemos las tipologías de las variables contenidas en el conjunto de datos (ver la figura anterior).

Como se anticipó, la primera variable es de tipo Factor, con dos niveles: No y Yes. Las 17 variables restantes son de tipo numérico. Como se anticipó en el documento, Conceptos de Redes Neuronales e Inteligencia Artificial, solo se pueden usar datos numéricos en el modelo, ya que una red neuronal es un modelo matemático con funciones de aproximación. Por lo tanto, tenemos un problema con la primera variable, Private.

No te preocupes, el problema se puede resolver fácilmente; simplemente conviértela en una variable numérica:

```
> Private = as.numeric(College$Private)-1
> data_scaled = cbind(Private,data_scaled)
```

En este sentido, la primera línea transforma la variable Private en numérica, mientras que la segunda línea de código se utiliza para reconstruir el conjunto de datos con esa variable y las 17 variables restantes, debidamente normalizadas. Para ello, utilizamos la función cbind, que toma una secuencia de argumentos vectoriales, matriciales o dataframes y los combina por columnas o filas, respectivamente:

```
> index = sample(1:nrow(data),round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])
```

Ha llegado el momento de dividir los datos para el entrenamiento y la prueba de la red. En la primera línea del código sugerido, el conjunto de datos se divide en 70:30, con la intención de utilizar el 70 % de los datos disponibles para entrenar la red y el 30 % restante para probarla. En la tercera línea, los datos del dataframe denominado data se subdividen en dos nuevos dataframes, denominados train_data y test_data:

```
> n = names(train_data)
> f <- as.formula(paste("Private ~", paste(n[!n %in% "Private"], collapse = " + ")))
```

En este fragmento de código, primero recuperamos todos los nombres de las variables mediante la función names. Esta función obtiene o establece el nombre de un objeto. A continuación, creamos la fórmula que usaremos para construir la red; para ello, utilizamos la función neuralnet para construir y entrenar la red. Hasta ahora, todo lo anterior se ha utilizado únicamente para preparar los datos. Ahora es el momento de construir la red:

```
> deep_net = neuralnet(f,data=train_data,hidden=c(5,3),linear.output=F)
```

Esta es la línea clave del código. Aquí se construye y entrena la red. Analicémosla en detalle. Habíamos previsto usar la biblioteca de redes neuronales para construir nuestra red neuronal

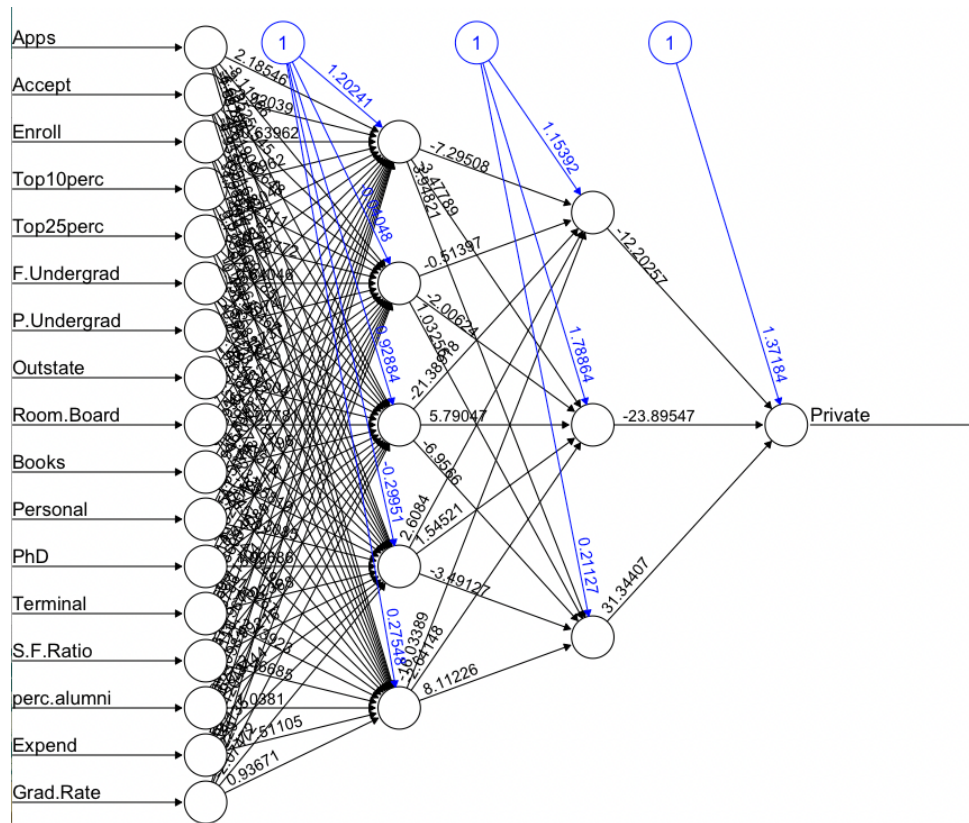
de capa oculta (DNN). Pero ¿qué ha cambiado con respecto a los casos en los que hemos construido una red de una sola capa oculta? Todo se juega en la configuración de argumentos ocultos.

Recuerda que el argumento oculto debe contener un vector de enteros que especifique el número de neuronas ocultas (vértices) en cada capa.

En nuestro caso, configuramos la capa oculta para que contenga el vector (5,3), que corresponde a dos niveles ocultos con cinco neuronas en la primera capa y tres neuronas en la segunda.

```
> plot(deep_net)
```

La línea anterior simplemente traza el diagrama de red, como se muestra en la siguiente figura:



Como podemos ver, la red está construida y entrenada, y solo tenemos que verificar su capacidad de predicción:

```
> predicted_data <- compute(deep_net, test_data[,2:18])
> print(head(predicted_data$net.result))
```

Para predecir los datos reservados para las pruebas, podemos utilizar el método de cálculo. Este método se utiliza para objetos de clase `nn`, generalmente generado por la función `neuralnet`.

Calcula las salidas de todas las neuronas para vectores de covariables arbitrarios específicos, dada una red neuronal entrenada. Es fundamental asegurarse de que el orden de las covariables sea el mismo en la nueva matriz o frame de datos que en la red neuronal original. Posteriormente, para visualizar las primeras líneas del resultado de la predicción, se utiliza la función `print`, que se muestra a continuación:

```

      [,1]
Abilene Christian University 0.2677593
Alaska Pacific University    1.0000000
Albion College               1.0000000
Alderson-Broaddus College    1.0000000
Alma College                 1.0000000
Anderson University          1.0000000

```

Como se puede observar, los resultados del pronóstico se proporcionan en forma de números decimales, que se aproximan a los valores de las dos clases esperadas (uno y cero), pero no asumen exactamente estos valores. Necesitamos asumir estos valores con precisión para poder compararlos con los valores actuales. Para ello, utilizaremos la función `sapply()` para redondearlos a cero o a una clase, y así evaluarlos con las etiquetas de prueba:

```
> predicted_data$net.result <- sapply(predicted_data$net.result,round,digits=0)
```

Como se anticipó, la función `sapply()` ha redondeado los resultados de la predicción en las dos clases disponibles. Ahora tenemos todo lo necesario para realizar una comparación y evaluar la DNN como herramienta de predicción:

```
> table(test_data$Private,predicted_data$net.result)
```

Para realizar una comparación, nos basamos en la **matriz de confusión**. Para construirla, simplemente utiliza la **función table**. De hecho, la función `table` utiliza los factores de clasificación cruzada para construir una tabla de contingencia de recuentos en cada combinación de niveles de factor.

La matriz de confusión es un diseño de tabla específico que permite visualizar el rendimiento de un algoritmo. Cada fila representa las instancias de una clase real, mientras que cada columna representa las instancias de una clase predicha. El término *matriz de confusión* se debe a que facilita la identificación de si el sistema confunde dos clases.

Veamos entonces los resultados obtenidos:

```

      0    1
0    51    8
1     8   166

```

Entendamos los resultados. Recordemos primero que en una matriz de confusión, los términos de la diagonal principal representan el número de predicciones correctas, es decir, el número de instancias de la clase predicha que coinciden con las instancias de la clase real. Parece que en nuestra simulación todo ha ido bien. De hecho, obtuvimos 51 ocurrencias de la clase 0 (No) y 166 de la clase 1 (Sí). Analicemos ahora los otros dos términos, que representan los errores cometidos por el modelo.

Como se define en el documento, Proceso de Aprendizaje en Redes Neuronales, 8 son FN y 8 son FP. En este sentido, recordamos que **FN representa el número de predicciones negativas que son positivas en los datos reales**, mientras que **FP representa el número de predicciones positivas que son negativas en los datos reales**. Podemos comprobarlo utilizando de nuevo la función de tabla:

```
> table(test_data$Private)
```

```

 0    1
59 174

```

Estos representan los resultados reales; en concreto, 59 resultados pertenecen a la clase 0 y 174 a la clase 1. Al sumar los datos de las filas de la matriz de confusión, obtenemos exactamente esos valores en los resultados reales:

```

> 51+8
[1] 59
> 8+166
[1] 174

```

Ahora, utilizamos de nuevo la función de table para **obtener las ocurrencias en los datos predichos**:

```
> table(predicted_data$net.result)
```

```

 0    1
59 174

```

Estos representan los resultados de la predicción; en concreto, 59 resultados pertenecen a la clase 0 y 174 a la clase 1. Al sumar los datos de las columnas de la matriz de confusión, obtenemos exactamente esos valores en los resultados reales:

```
> 51 + 8
[1] 59
> 8 + 166
[1] 174
```

En este punto, calculamos la precisión de la simulación utilizando los datos de la matriz de confusión. Recordemos que la precisión se define mediante la siguiente ecuación:

$$\text{Accuracy} = \frac{TP + FN}{P + N} = \frac{TP + FN}{TP + FN + FP + FN}$$

Donde:

TP = True Positive
 TN = True Negative
 FP = False Positive
 FN = False Negative

Analicemos esto en el siguiente ejemplo de código:

```
> Acc = (51 + 166)/(51 + 166 + 8 + 8)
> Acc
[1] 0.9313305
```

Obtuvimos una precisión de alrededor del 93 %, lo que confirma que nuestro modelo puede predecir datos con buenos resultados.

Entrenamiento y modelado de una DNN con H2O

En esta sección, abordaremos un ejemplo de entrenamiento y modelado de una DNN con **h2o**. **h2o es una plataforma de aprendizaje automático e IA escalable**, en memoria y de código abierto que se utiliza para crear modelos con grandes conjuntos de datos e implementar predicciones con métodos de alta precisión.

La biblioteca h2o se ha adaptado a gran escala en numerosas organizaciones para implementar la ciencia de datos y proporcionar una plataforma para crear productos de datos. h2o puede ejecutarse en portátiles individuales o en grandes clústeres de servidores escalables de alto rendimiento. Funciona muy rápido, aprovechando los avances en la arquitectura de la máquina y el procesamiento de GPU. Cuenta con implementaciones de alta precisión de aprendizaje profundo, redes neuronales y otros algoritmos de aprendizaje automático.

Como se mencionó anteriormente, el paquete h2o de R incluye funciones para crear regresión lineal general, K-medias, Naive Bayes, PCA, bosques y aprendizaje profundo (modelos de redes neuronales multicapa). El paquete h2o es un paquete externo a CRAN y se crea en Java. Está disponible para diversas plataformas.

Instalaremos h2o en R con el siguiente código:

```
> install.packages("h2o")
```

Obtenemos los siguientes resultados (en MacOS):

```
probando la URL 'https://cran.mirror.rafal.ca/bin/macosx/big-sur-x86_64/contrib/4.5/h2o_3.44.0.3.tgz'
Content type 'application/x-gzip' length 266906239 bytes (254.5 MB)
=====
downloaded 254.5 MB
```

Para probar el paquete, analicemos el siguiente ejemplo que utiliza el popular conjunto de datos **Irisdataset**. Me refiero al conjunto de datos de la flor de iris, un conjunto de datos multivariante introducido por el estadístico y biólogo británico Ronald Fisher en su artículo de 1936 “El uso de múltiples mediciones en problemas taxonómicos” como ejemplo de análisis discriminante lineal.

El conjunto de datos contiene **50 muestras** de cada una de las tres especies de iris (Iris setosa, Iris virginica e Iris versicolor). Se midieron **cuatro características** de cada muestra: la longitud y la anchura de los sépalos y pétalos, en centímetros.

Contiene las siguientes variables:

- Sepal.Length en centímetros
- Sepal.Width en centímetros
- Petal. Length en centímetros
- Petal. Width en centímetros
- Class: setosa, versicolor, virginica

La siguiente figura muestra una representación compacta de la estructura del conjunto de datos de iris:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
14	4.3	3.0	1.1	0.1	setosa
15	5.8	4.0	1.2	0.2	setosa
16	5.7	4.4	1.5	0.4	setosa
17	5.4	3.9	1.3	0.4	setosa
18	5.1	3.5	1.4	0.3	setosa
19	5.7	3.8	1.7	0.3	setosa
20	5.1	3.8	1.5	0.3	setosa
21	5.4	3.4	1.7	0.2	setosa
22	5.1	3.7	1.5	0.4	setosa
23	4.6	3.6	1.0	0.2	setosa
24	5.1	3.3	1.7	0.5	setosa
25	4.8	3.4	1.9	0.2	setosa
26	5.0	3.0	1.6	0.2	setosa
27	5.0	3.4	1.6	0.4	setosa
28	5.2	3.5	1.5	0.2	setosa
29	5.2	3.4	1.4	0.2	setosa
30	4.7	3.2	1.6	0.2	setosa

Queremos construir un clasificador que, según el tamaño del sépalo y el pétalo, pueda clasificar las especies de flores:

```
> library(h2o)
> cl=h2o.init(max_mem_size = "2G",
+   nthreads = 2,
+   ip = "localhost",
+   port = 54321)
> data(iris)
> summary(iris)
> iris_d1 <- h2o.deeplearning(1:4,5,
+   as.h2o(iris),hidden=c(5,5),
```



```

+ export_weights_and_biases=T)
> iris_d1
> plot(iris_d1)
> h2o.weights(iris_d1, matrix_id=1)
> h2o.weights(iris_d1, matrix_id=2)
> h2o.weights(iris_d1, matrix_id=3)
> h2o.biases(iris_d1, vector_id=1)
> h2o.biases(iris_d1, vector_id=2)
> h2o.biases(iris_d1, vector_id=3)
> # plot weights connecting `Sepal.Length` to first hidden neurons
> plot(as.data.frame(h2o.weights(iris_d1, matrix_id=1))[,1])

```

Ahora, revisemos el código para comprender cómo aplicar el paquete h2o para resolver un problema de reconocimiento de patrones.

Antes de continuar, es necesario especificar que ejecutar h2o (uso la versión 3.44.0.3) en R requiere Java. Verifica previamente la versión de Java instalada en tu equipo y, eventualmente, descarga la versión 8 mínimamente de Java y hasta la versión 17.

Además, el paquete h2o incluye algunos paquetes necesarios; por lo tanto, para instalarlo correctamente, recuerda instalar las siguientes dependencias, todas disponibles en CRAN (de hecho la misma instalación los descarga e instala):

- RCurl
- bitops
- rjson
- jsonlite
- statmod
- tools

Después de instalar correctamente el paquete h2o, podemos proceder a cargar la biblioteca:

```
> library(h2o)
```

Este comando carga la biblioteca en el entorno R. Se devuelven algunos mensajes, de acuerdo a tu sistema operativo.

Seguimos las instrucciones del prompt de R:

```

> cl=h2o.init(max_mem_size = "2G",
+ nthreads = 2,
+ ip = "localhost",
+ port = 54321)

```

La función `h2o.init` inicia el motor `h2o` con una memoria máxima de 2 GB y dos núcleos paralelos. La consola de `h2o` se inicializa y, al ejecutar este script, obtenemos los siguientes mensajes:

Connection successful!

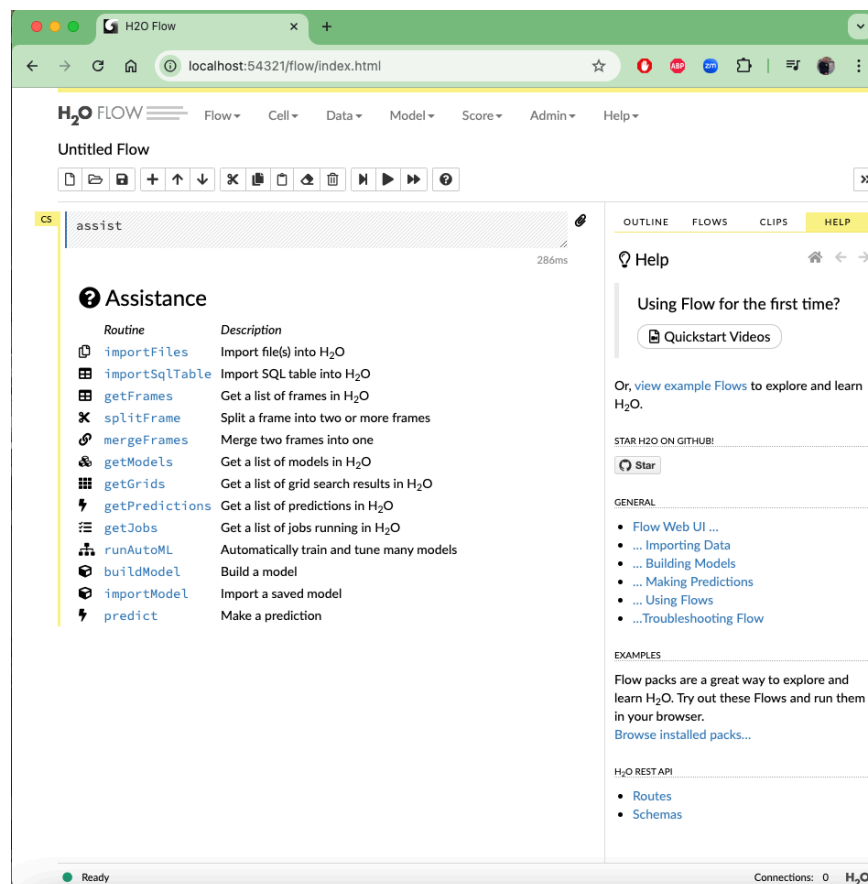
R is connected to the H2O cluster:

```

H2O cluster uptime:      19 minutes 49 seconds
H2O cluster timezone:    America/Mexico_City
H2O data parsing timezone: UTC
H2O cluster version:     3.44.0.3
H2O cluster version age:  1 year, 3 months and 25 days
H2O cluster name:        H2O_started_from_R_movilesFCC2025_tnb710
H2O cluster total nodes: 1
H2O cluster total memory: 1.97 GB
H2O cluster total cores: 4
H2O cluster allowed cores: 2
H2O cluster healthy:     TRUE
H2O Connection ip:       localhost
H2O Connection port:     54321
H2O Connection proxy:    NA
H2O Internal Security:   FALSE
R Version:               R version 4.5.0 (2025-04-11)

```

Una vez iniciado `h2o`, la consola se puede ver desde cualquier navegador apuntando a `localhost:54321`. La biblioteca `h2o` se ejecuta en una JVM y la consola permite:



La consola es intuitiva y proporciona una interfaz para interactuar con el motor h2o. Podemos entrenar y probar modelos, así como ejecutar predicciones sobre ellos. El primer cuadro de texto, denominado CS, permite introducir rutinas para su ejecución. El comando `assist` muestra la lista de rutinas disponibles. Analicemos el siguiente código de ejemplo.

```
> data(iris)
> summary(iris)
```

El primer comando carga el conjunto de datos iris, contenido en la biblioteca de conjuntos de datos, y lo guarda en un dataframe determinado. A continuación, utilizamos la función `summary` para generar resúmenes de los resultados del conjunto de datos.

La función invoca métodos específicos que dependen de la clase del primer argumento. Los resultados se muestran a continuación:

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100	setosa :50
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300	versicolor:50
Median :5.800	Median :3.000	Median :4.350	Median :1.300	virginica :50
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199	
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800	
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500	

Analicemos las siguientes líneas de código:

```
> iris_d1 <- h2o.deeplearning(1:4,5,
+ as.h2o(iris),hidden=c(5,5),
+ export_weights_and_biases=T)
```

La función `h2o.deeplearning` es una función importante dentro de `h2o` y puede utilizarse para diversas operaciones. Esta función construye un modelo de red neuronal profunda (DNN) mediante CPU y una red neuronal artificial (RNA) multicapa de propagación hacia adelante (feedforward) en un `H2OFrame`. El argumento `hidden` se utiliza para establecer el número de capas ocultas y el número de neuronas para cada capa oculta.

En nuestro caso, hemos configurado una DNN con dos capas ocultas y 5 neuronas para cada una. Finalmente, el parámetro `export_weights_and_biases` indica que los pesos y sesgos pueden almacenarse en el `H2OFrame` y accederse a ellos como a otros frames de datos para su posterior procesamiento.

Antes de continuar con el análisis del código, conviene hacer una aclaración. El lector atento podría preguntarse en base a qué evaluación hemos elegido el número de capas ocultas y el número de neuronas para cada capa oculta. Desafortunadamente, no existe una regla precisa

ni siquiera una fórmula matemática que nos permita determinar qué números son apropiados para cada problema específico. Esto se debe a que cada problema es diferente y cada red se aproxima a un sistema de forma distinta. Entonces, ¿qué marca la diferencia entre un modelo y otro? La respuesta es obvia y, una vez más, muy clara: la experiencia del investigador.

El consejo que se puede dar, basado en la amplia experiencia en análisis de datos, es probar, probar y volver a probar. El secreto de la actividad experimental reside precisamente en eso. En el caso de las redes neuronales, esto implica intentar configurar diferentes redes y luego verificar su rendimiento.

Por ejemplo, en nuestro caso, podríamos haber partido de una red con dos capas ocultas y 100 neuronas por capa oculta, reducir progresivamente esos valores y llegar a los que se proponen en el ejemplo. Este procedimiento se puede automatizar mediante el uso de las estructuras iterativas de R.

Sin embargo, se pueden mencionar algunas cosas, por ejemplo, **para la elección óptima del número de neuronas**, que necesitamos saber:

- **Un número pequeño de neuronas** generará un alto error en el sistema, ya que los factores predictivos podrían ser demasiado complejos para que un número pequeño de neuronas los capture.
- **Un número grande de neuronas** se sobreajustará a los datos de entrenamiento y no se generalizará correctamente.
- El número de neuronas en cada capa oculta debe estar entre el tamaño de la capa de entrada y la de salida, potencialmente la media.
- El número de neuronas en cada capa oculta no debe exceder el doble del número de neuronas de entrada, ya que probablemente el sobreajuste sea considerable en este punto.

Dicho esto, volvamos al código:

```
> iris_d1
```

En el prompt de R, este comando imprime una breve descripción de las características del modelo que acabamos de crear, como se muestra en la siguiente figura:

Al analizar detenidamente la figura, podemos distinguir claramente los detalles del modelo junto con la matriz de confusión.

Veamos ahora cómo se desarrolló el proceso de entrenamiento:

```
Model Details:
=====

H2OMultinomialModel: deeplearning
Model ID: DeepLearning_model_R_1744683976429_1
Status of Neuron Layers: predicting Species, 3-class classification, multinomial distribution, CrossEntropy loss, 73 weights/biases,
4.7 KB, 1,500 training samples, mini-batch size 1
layer units      type dropout      l1      l2 mean_rate rate_rms momentum mean_weight weight_rms mean_bias bias_rms
1      1      4      Input 0.00 %      NA      NA      NA      NA      NA      NA      NA      NA
2      2      5 Rectifier 0.00 % 0.000000 0.000000 0.002114 0.000932 0.000000 0.202210 0.460674 0.584752 0.106385
3      3      5 Rectifier 0.00 % 0.000000 0.000000 0.000740 0.000388 0.000000 0.000005 0.403754 0.979237 0.138141
4      4      3 Softmax      NA 0.000000 0.000000 0.004411 0.006654 0.000000 -0.252285 1.995078 -0.002404 0.133110

H2OMultinomialMetrics: deeplearning
** Reported on training data. **
** Metrics reported on full training frame **

Training Set Metrics:
=====

Extract training frame with `h2o.getFrame("iris_sid_bfcd_1")`
MSE: (Extract with `h2o.mse`) 0.07480726
RMSE: (Extract with `h2o.rmse`) 0.2735092
Logloss: (Extract with `h2o.logloss`) 0.2620216
Mean Per-Class Error: 0.08
AUC: (Extract with `h2o.auc`) NaN
AUCPR: (Extract with `h2o.aucpr`) NaN
Confusion Matrix: Extract with `h2o.confusionMatrix(<model>,train = TRUE)`

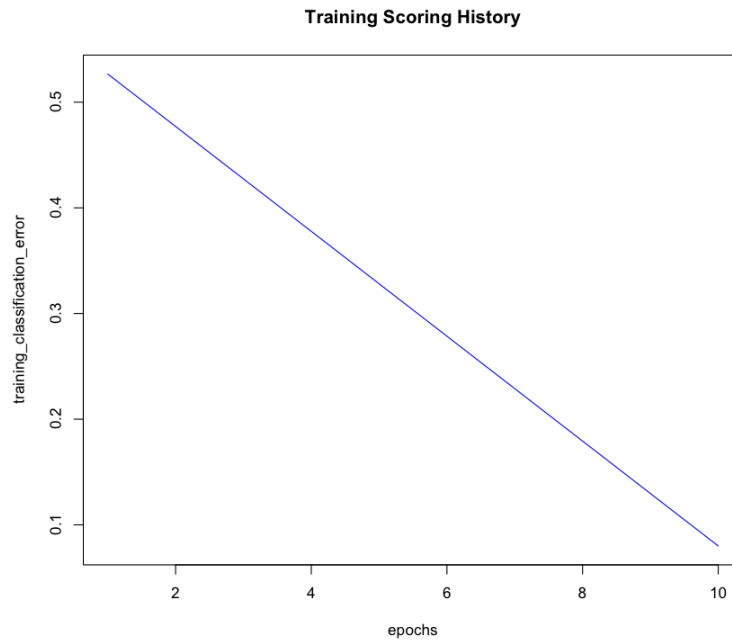
Confusion Matrix: Row labels: Actual class; Column labels: Predicted class
              setosa versicolor virginica Error Rate
setosa        50           0           0 0.0000 = 0 / 50
versicolor     0          45           5 0.1000 = 5 / 50
virginica       0           7          43 0.1400 = 7 / 50
Totals         50          52          48 0.0800 = 12 / 150

Hit Ratio Table: Extract with `h2o.hit_ratio_table(<model>,train = TRUE)`
=====
Top-3 Hit Ratios:
k hit_ratio
1 1 0.920000
2 2 1.000000
3 3 1.000000
```

```
> plot(iris_d1)
```

El método plot se centra en el tipo de modelo h2o para seleccionar el historial de puntuación correcto. Los argumentos se limitan a lo disponible en el historial de puntuación para un tipo de modelo en particular, como se muestra en la siguiente figura:

En este gráfico se muestran los errores de clasificación de entrenamiento en función de las épocas. Podemos observar que el gradiente desciende y los errores disminuyen a medida que avanzamos en las épocas.



El número de iteraciones (transmisiones, streamed) del conjunto de datos puede ser fraccional. El valor predeterminado es 10:

```
> h2o.weights(iris_d1, matrix_id=1)
> h2o.weights(iris_d1, matrix_id=2)
> h2o.weights(iris_d1, matrix_id=3)
> h2o.biases(iris_d1, vector_id=1)
> h2o.biases(iris_d1, vector_id=2)
> h2o.biases(iris_d1, vector_id=3)
```

Las últimas seis líneas del código simplemente imprimen un breve resumen de las ponderaciones y los sesgos para las tres especies de flor de iris, como se muestra a continuación:

```
> h2o.weights(iris_d1, matrix_id=1)

  Sepal.Length Sepal.Width Petal.Length Petal.Width
1    0.1789352    0.2662688    1.1472908    0.6277370
2    0.1053760    0.5060248   -0.1988446    0.6095566
3    0.5827854   -0.4895157   -0.1524225   -0.2209050
4    0.3846286    0.5227697    0.1248523   -0.7597406
5    0.6080559   -0.5837346    0.3733365    0.4117365

[5 rows x 4 columns]
```

```
> h2o.weights(iris_d1, matrix_id=2)
```

	C1	C2	C3	C4	C5
1	0.5985000	0.165896967	0.4885534	0.4839202	-0.7153376
2	-0.2664635	-0.444854438	-0.5184130	-0.5404316	0.1577795
3	0.1570279	0.176452786	0.4824809	0.1234057	-0.1543938
4	-0.4055207	0.001228933	0.0647134	0.5317967	-0.6673959
5	-0.1920279	0.206376016	0.3286178	0.4085667	-0.4703650

[5 rows x 5 columns]

```
> h2o.biases(iris_d1, vector_id=1)
```

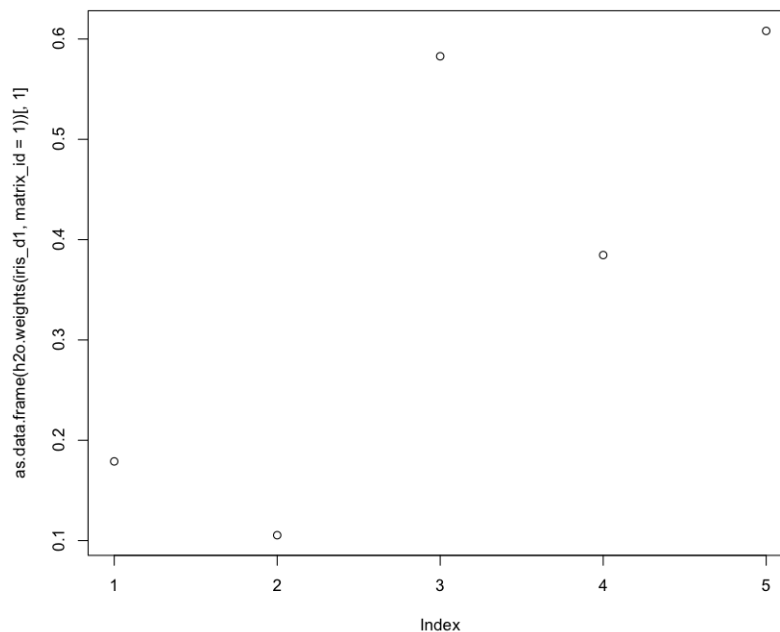
	C1
1	0.4922919
2	0.6540384
3	0.6230461
4	0.4319486
5	0.7224345

[5 rows x 1 column]

Por razones de espacio, nos hemos limitado a ver las ponderaciones y los sesgos solo para la especie setosa. En el siguiente código, volvemos a utilizar la función plot:

```
> plot(as.data.frame(h2o.weights(iris_d1, matrix_id=1))[,1])
```

Este comando grafica los pesos de las neuronas de la primera capa oculta frente a la longitud de los sépalos, como se muestra en la siguiente figura:



Ahora, dediquemos un tiempo al análisis de los resultados; en particular, recuperamos la matriz de confusión que acabamos de ver en la pantalla de resumen del modelo, mostrada

anteriormente. Para invocar la matriz de confusión, podemos utilizar la función `h2o.confusionMatrix`, como se muestra en el siguiente ejemplo de código, que recupera matrices de confusión individuales o múltiples de los objetos `h2o`.

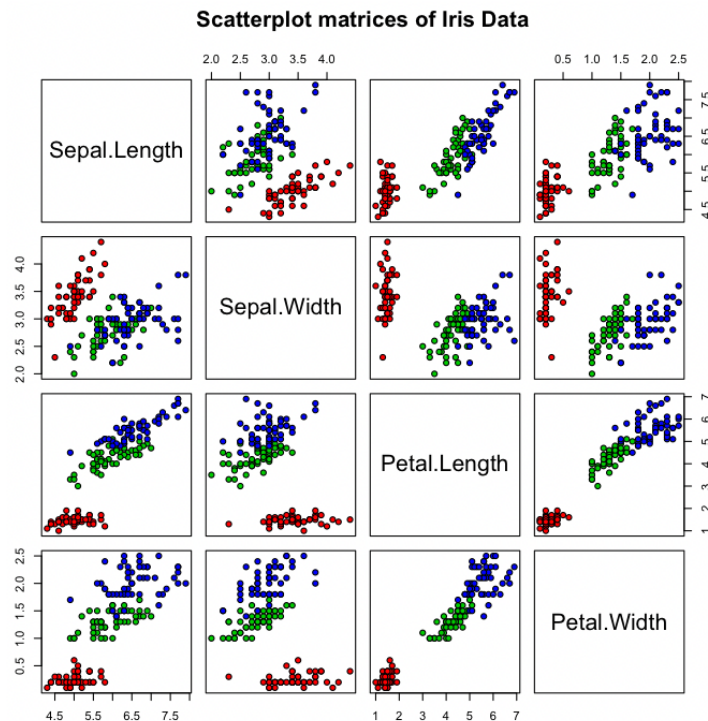
```
> h2o.confusionMatrix(iris_d1)
```

```
Confusion Matrix: Row labels: Actual class; Column labels: Predicted class
              setosa versicolor virginica Error      Rate
setosa         50          0           0 0.0000 =  0 / 50
versicolor     0          45          5 0.1000 =  5 / 50
virginica       0          7         43 0.1400 =  7 / 50
Totals         50         52         48 0.0800 = 12 / 150
```

Del análisis de la matriz de confusión, se observa que el modelo logra clasificar correctamente las tres especies florales con solo doce errores. Estos errores se reparten equitativamente entre solo dos especies: *versicolor* y *virginica*. Sin embargo, la especie *setosa* se clasifica correctamente en las 50 apariciones. ¿Por qué ocurre esto? Para comprenderlo, analicemos los datos iniciales. En el caso de datos multidimensionales, la mejor manera de hacerlo es trazar una matriz de diagrama de dispersión con las variables seleccionadas en un conjunto de datos:

```
> pairs(iris[1:4], main = "Scatterplot matrices of Iris Data", pch = 21, bg = c("red",
+   "green3", "blue")[unclass(iris$Species)])
```

Los resultados se muestran en la siguiente figura:



Analicemos en detalle el gráfico propuesto. Las variables se escriben en una línea diagonal de arriba a la izquierda a abajo a la derecha. A continuación, cada variable se compara con las demás. Por ejemplo, el segundo recuadro de la primera columna es un diagrama de dispersión individual de Sepal.Length versus Sepal.Width, con Sepal.Length como eje X y Sepal.Width como eje Y. Este mismo gráfico se replica en el primer gráfico de la segunda columna. En esencia, los recuadros de la esquina superior derecha de todo el diagrama de dispersión son imágenes especulares de los gráficos de la esquina inferior izquierda.

Del análisis de la figura, se puede observar que las especies versicolor y virginica presentan límites superpuestos. Esto nos lleva a comprender que el intento del modelo de clasificarlas cuando se encuentran en esa zona puede causar errores. Podemos observar lo que ocurre con la especie setosa, que, en cambio, presenta límites muy distantes de otras especies florales sin ningún error de clasificación.

Dicho esto, evaluamos la precisión del modelo al clasificar las especies florales según el tamaño de los pétalos y sépalos:

```
> h2o.hit_ratio_table(iris_d1)
```

```
Top-3 Hit Ratios:
```

```
  k hit_ratio
1 1  0.920000
2 2  1.000000
3 3  1.000000
```

Los resultados muestran que la simulación, basada en la primera hipótesis, clasificó las especies con un 92 % de precisión. Diríamos que es un buen resultado; el modelo se ajustó muy bien a los datos. Pero ¿cómo podemos medir esta característica? **Un método para encontrar un mejor ajuste es calcular el coeficiente de determinación (R^2).** Para calcular R^2 en h2o, podemos utilizar el método h2o.r2:

```
> h2o.r2(iris_d1)
[1] 0.8877891
```

Ahora entendamos qué hemos calculado y cómo interpretar el resultado. El R cuadrado mide la capacidad de un modelo para predecir los datos y se encuentra entre cero y uno. Cuanto mayor sea el valor del coeficiente de determinación, mejor será el modelo para predecir los datos.

Obtuvimos un valor de 0.89, lo que, según lo mencionado, es un excelente resultado. Para confirmarlo, debemos compararlo con el resultado de otro modelo de simulación. Por lo

tanto, construimos un modelo de regresión lineal basado en los mismos datos, es decir, el conjunto de datos iris.

Para construir un modelo de regresión lineal, podemos usar la función `glm`. Esta función se utiliza para ajustar modelos lineales generalizados, especificados mediante una descripción simbólica del predictor lineal y una descripción de la distribución del error:

```
> m=iris.lm <- h2o.glm(x=2:5,y=1,training_frame=as.h2o(iris))
```

Ahora calculamos el coeficiente de determinación del modelo:

```
> h2o.r2(m)
[1] 0.8667852
```

Ahora podemos comparar el modelo basado en DNN y el modelo de regresión lineal. La DNN había proporcionado un valor R-cuadrado de 0.89, mientras que el modelo de regresión resultante arrojó un valor R-cuadrado de 0.87. Es evidente que la DNN ofrece un rendimiento mucho mejor.

Finalmente, puede ser útil analizar los parámetros importantes para un especialista en redes neuronales, como se muestra en la siguiente tabla:

Argumento	Descripción
x	Un vector que contiene los nombres o índices de las variables predictoras que se utilizarán en la construcción de un modelo. Si falta x, se utilizan todas las columnas excepto y.
y	El nombre de la variable de respuesta en un modelo. Si los datos no contienen un encabezado, este es el índice de la primera columna, que aumenta de izquierda a derecha (la respuesta debe ser un entero o una variable categórica).
model_id	Este es el id de destino de un modelo; se genera automáticamente si no se especifica.
standardize	Es una función lógica. Si está habilitada, estandariza automáticamente los datos. Si está deshabilitada, el usuario debe proporcionar datos de entrada correctamente escalados. El valor predeterminado es TRUE.
activation	Es una función de activación. Debe ser una de las siguientes: Tanh, TanhWithDropout, Rectifier, RectifierWithDropout, Maxout o MaxoutWithDropout. El valor predeterminado es Rectifier.
hidden	Este argumento especifica el tamaño de las capas ocultas (por ejemplo, [100, 100]). Su valor predeterminado es [200, 200].
epochs	El número de iteraciones (streamed) del conjunto de datos puede ser fraccionario. Su valor predeterminado es 10.
adaptive_rate	Es un argumento lógico que especifica la tasa de aprendizaje adaptativo. Su valor predeterminado es TRUE.

rho	Describe el factor de decaimiento temporal de la tasa de aprendizaje adaptativo (similar a actualizaciones anteriores). Su valor predeterminado es 0.99.
rate_annealing	La tasa de aprendizaje de annealing se obtiene mediante $\text{rate}/(1 + \text{rate_annealing} * \text{samples})$. Su valor predeterminado es 1e-06.
rate_decay	Este es el factor de decaimiento de la tasa de aprendizaje entre capas (capa N: $\text{rate} * \text{rate_decay} ^ (n - 1)$). Su valor predeterminado es 1.
input_dropout_ratio	Tasa de abandono de la capa de entrada (puede mejorar la generalización; prueba con 0.1 o 0.2). Su valor predeterminado es 0.
hidden_dropout_ratios	Las tasas de abandono de la capa oculta pueden mejorar la generalización. Especifica un valor por capa oculta. Su valor predeterminado es 0.5.
l1	La regularización L1 puede aumentar la estabilidad y mejorar la generalización, y hace que muchos pesos se conviertan en 0. Su valor predeterminado es 0.
l2	La regularización L2 puede aumentar la estabilidad y mejorar la generalización, y hace que muchos pesos sean pequeños. Su valor predeterminado es 0.
initial_weights	Esta es una lista de los ID de H2OFrame con los que se inicializan los vectores de pesos de este modelo.
initial_biases	Es una lista de los ID de H2OFrame Para inicializar los vectores de sesgo de este modelo.
loss	La función de pérdida debe ser Automatic, CrossEntropy (Entropía Cruzada), Quadratic, Huber, Absolute o Quantile. Su valor predeterminado es Automatic.
distribution	La función de distribución debe ser AUTO, bernoulli, multinomial, gaussian, poisson, gamma, tweedie, laplace, quantile o huber. Su valor predeterminado es AUTO.
score_training_samples	Es el número de muestras del conjunto de entrenamiento para la puntuación (0 para todas). Su valor predeterminado es 10000.
score_validation_samples	Es el número de muestras del conjunto de validación para la puntuación (0 para todas). Su valor predeterminado es 0.
classification_stop	El criterio de detención para la fracción de error de clasificación en los datos de entrenamiento (-1 para deshabilitar). Su valor predeterminado es 0.
representation_stop	Es el criterio de detención para el error de regresión (MSE) en los datos de entrenamiento (-1 para deshabilitar). Su valor predeterminado es 1e-06.
stoppping_rounds	Detención anticipada basada en la convergencia de la métrica de detención (stoppping_metric). Detenerse si el promedio móvil simple de longitud k de la métrica de detención no mejora para los eventos de puntuación k:=stoppping_rounds (0 para deshabilitar). El valor predeterminado es 5.
max_runtime_secs	Es el tiempo de ejecución máximo permitido en segundos para el entrenamiento del modelo. Usa 0 para deshabilitarlo. El valor predeterminado es 0.
diagnostics	Habilita el diagnóstico de capas ocultas. El valor predeterminado es TRUE.
fast_mode	Habilita el modo rápido (aproximación menor en retropropagación). El valor predeterminado es TRUE.

replicate_training_data	Replica todo el conjunto de datos de entrenamiento en cada nodo para un entrenamiento más rápido en conjuntos de datos pequeños. El valor predeterminado es TRUE.
single_node_mode	Se ejecuta en un solo nodo para ajustar los parámetros del modelo. El valor predeterminado es FALSE.
shuffle_training_data	Habilita la reorganización de los datos de entrenamiento (recomendado si los datos de entrenamiento se replican y train_samples_per_iteration es cercano a #nodes x #rows, o si se utilizan balance_classes). El valor predeterminado es FALSE.
missing_values_handling	El manejo de valores faltantes debe ser MeanImputation u Skip. El valor predeterminado es MeanImputation.
quiet_mode	Habilita el modo silencioso para reducir la salida estándar. El valor predeterminado es FALSE.
verbose	Imprime el historial de puntuación en la consola (métricas por árbol para GBM, DRF y XGBoost; métricas por época para aprendizaje profundo). El valor predeterminado es FALSE.
autoencoder	El autocodificador lógico tiene el valor FALSO predeterminado.
export_weights_and_biases	Si se exportan los pesos y sesgos de la red neuronal a H2OFrame. El valor predeterminado es FALSE.
mini_batch_size	Tamaño del minilote (un tamaño más pequeño proporciona un mejor ajuste, mientras que un tamaño más grande permite una mayor velocidad y una mejor generalización). El valor predeterminado es 1.

Existen otras funciones relacionadas para R con h2o para aprendizaje profundo. Algunas útiles se enumeran a continuación:

Función	Descripción
predict.H2Omodel	Devuelve un objeto H2OFrame con probabilidades y predicciones predeterminadas.
h2o.deepwater	Construye un modelo de aprendizaje profundo utilizando múltiples backends de GPU nativos. Construye una DNN en H2OFrame que contiene diversas fuentes de datos.
as.data.frame.H2OFrame	Convierte H2OFrame en un dataframe.
h2o.confusionMatrix	Muestra la matriz de confusión de un modelo de clasificación.
print.H2OFrame	Imprime H2OFrame.
h2o.saveModel	Guarda un objeto de modelo h2o en el disco.
h2o.importFile	Importa un archivo a h2o.